

Objective: To design a 3 Bit Binary Adder using two three bit inputs.

Equipment: Vivado 2017-4 software used to program a BASYS 3 circuit board.

Procedure/Design: The adder takes in two three bit inputs, having two intermediate outputs: the carry ins of the adder and two actual outputs: the carry out and the sum. Represented by the truth table and karnaugh maps below:

Experiment 3 Pre Lab.

3 bit Full Adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum = $A \oplus B \oplus C_{in}$

A	B	Cin	Sum
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Cout = $AB + C_{in}AB + A\bar{B} + \bar{A}B$

A	B	Cin	Cout
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Check for pattern

$AB + C_{in}AB + A\bar{B} + \bar{A}B$

$AB + C_{in}(AB)$

1 1
1 1
0 0
0 0

Source code:

Source code for `AdderThreeBit.v` in Verilog:

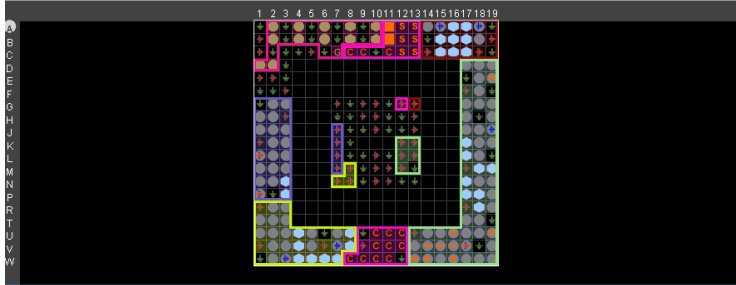
```

1  `timescale 1ns / 1ps
2
3
4  module AdderThreeBit(
5      input wire [2:0] X,
6      input wire [2:0] Y,
7      output wire C1,
8      output wire C2,
9      output wire [2:0] S,
10     output wire Cout
11 );
12
13
14     assign S[0]=X[0]^Y[0];
15     assign C1 = X[0]&Y[0];
16     assign S[1]=X[1]^Y[1]^C1;
17     assign C2 = (X[1]&Y[1]) | (C1 & Y[1]) | (C1 & X[1]);
18     assign S[2] = X[2]^Y[2]^C2;
19
20     assign Cout=(C2&X[2]) | (Y[2]&C2) | (X[2]&Y[2]);
21 endmodule
22

```

Schematic view showing the implementation of the 3-bit adder using logic gates (RTL_AND, RTL_OR, RTL_XOR) and signals (Cout, C1, C2, S[2:0]).

Package x Device x AdderThreeBit.v x ? □



IO Ports

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	IO Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination
S[0]	OUT		U19	✓	14	LVTMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
S[1]	OUT		E19	✓	14	LVTMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
S[2]	OUT		U16	✓	14	LVTMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
X[0]	IN		W16	✓	14	LVTMOS33*	3.300				NONE	NONE
X[1]	IN		V16	✓	14	LVTMOS33*	3.300				NONE	NONE
X[2]	IN		V17	✓	14	LVTMOS33*	3.300				NONE	NONE
Y[0]	IN		W13	✓	14	LVTMOS33*	3.300				NONE	NONE
Y[1]	IN		W14	✓	14	LVTMOS33*	3.300				NONE	NONE
Y[2]	IN		V15	✓	14	LVTMOS33*	3.300				NONE	NONE
C1	OUT		L1	✓	35	LVTMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
C2	OUT		P1	✓	35	LVTMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
Cout	OUT		V14	✓	14	LVTMOS33*	3.300		12	SLOW	NONE	FP_VTT_50

Test Bench:

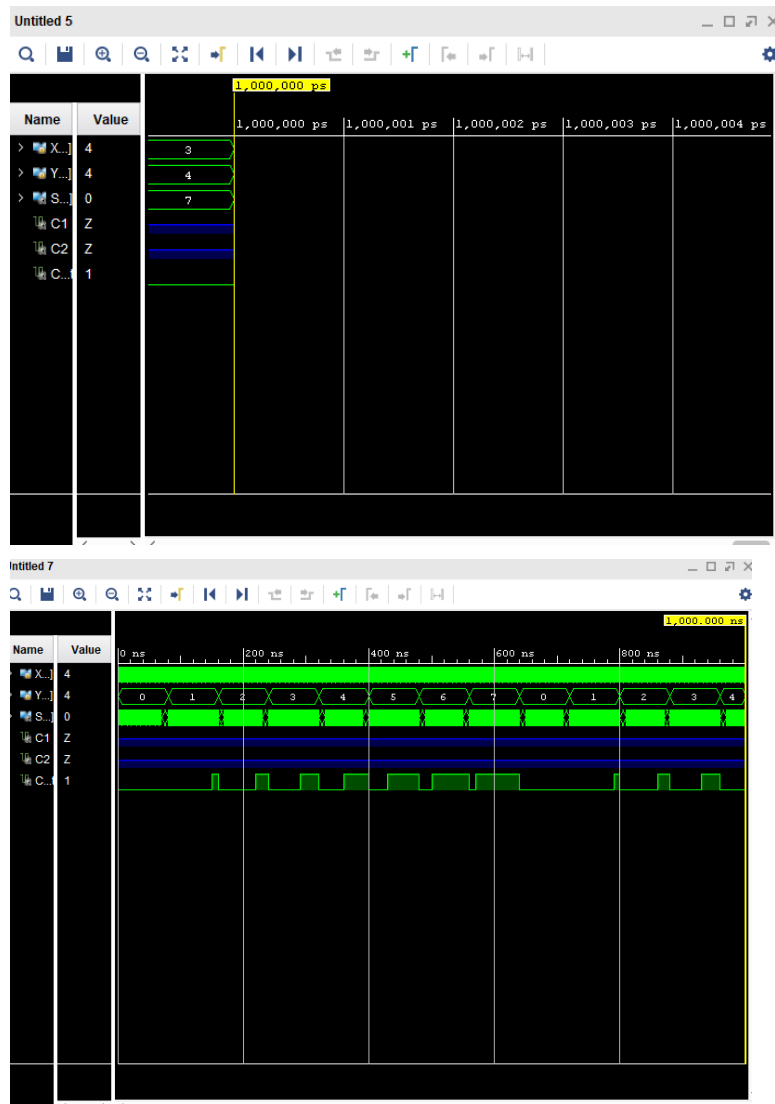
adderThreeBit_sim.v ? □

/n/266542/Desktop/EEE3342/AdderThreeBit/AdderThreeBit.srcs/sim_1/new/AdderThreeBit_sim.v

```

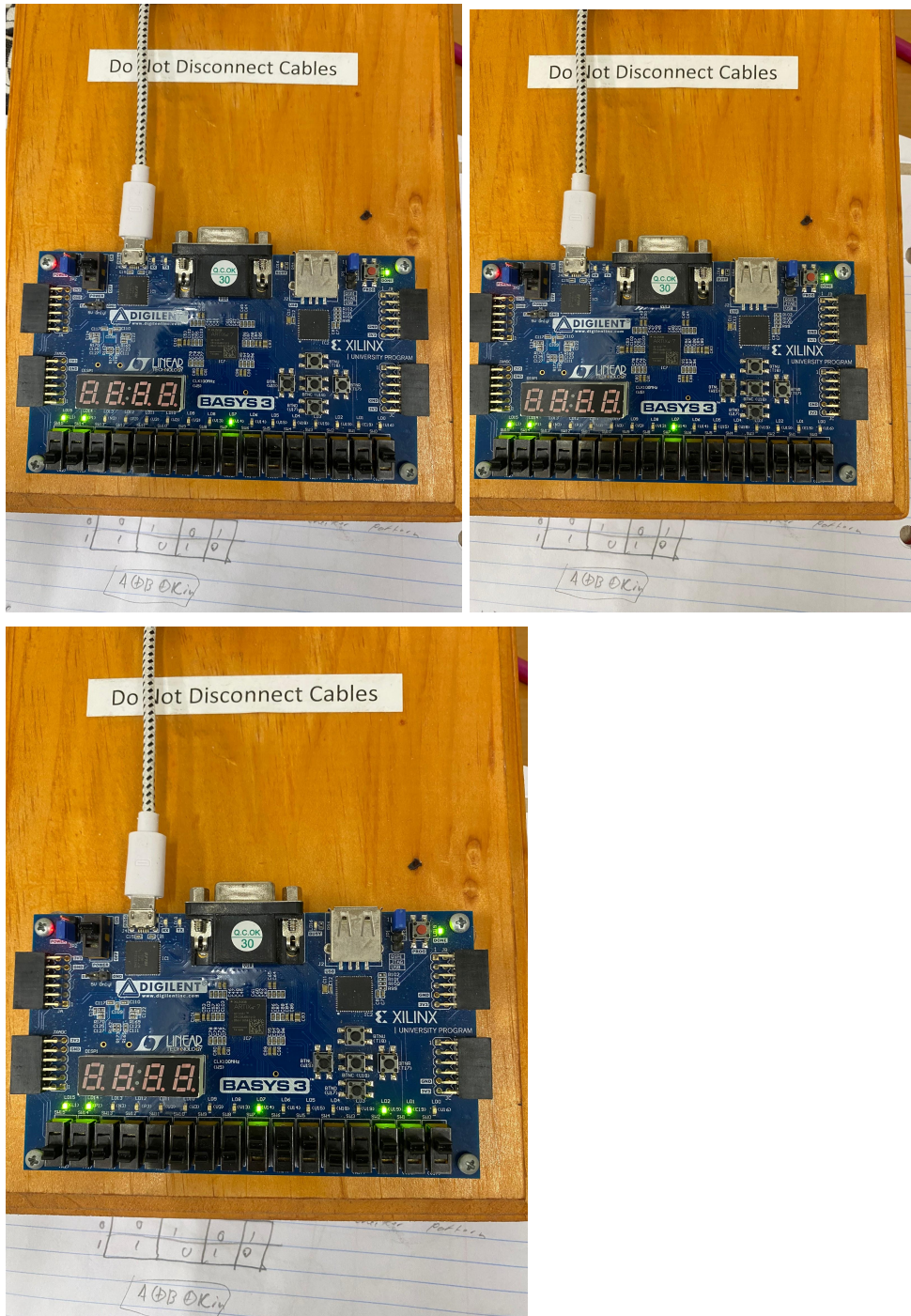
3
4 module AdderThreeBit_sim();
5   reg [2:0] X_t;
6   reg [2:0] Y_t;
7   wire [2:0] S_t;
8   wire C1;
9   wire C2;
10  wire Cout_t;
11  AdderThreeBit UUT(
12    .X(X_t),
13    .Y(Y_t),
14    .S(S_t),
15    .Cout(Cout_t)
16  );
17
18  initial
19  begin
20    X_t[2:0]=3'b000;
21    Y_t[2:0]=3'b000;
22  end
23
24  always #10 X_t[0]=~X_t[0];
25  always #20 X_t[1]=~X_t[1];
26  always #40 X_t[2]=~X_t[2];
27
28  always #80 Y_t[0]=~Y_t[0];
29  always #160 Y_t[1]=~Y_t[1];
30  always #320 Y_t[2]=~Y_t[2];
31
32 endmodule

```

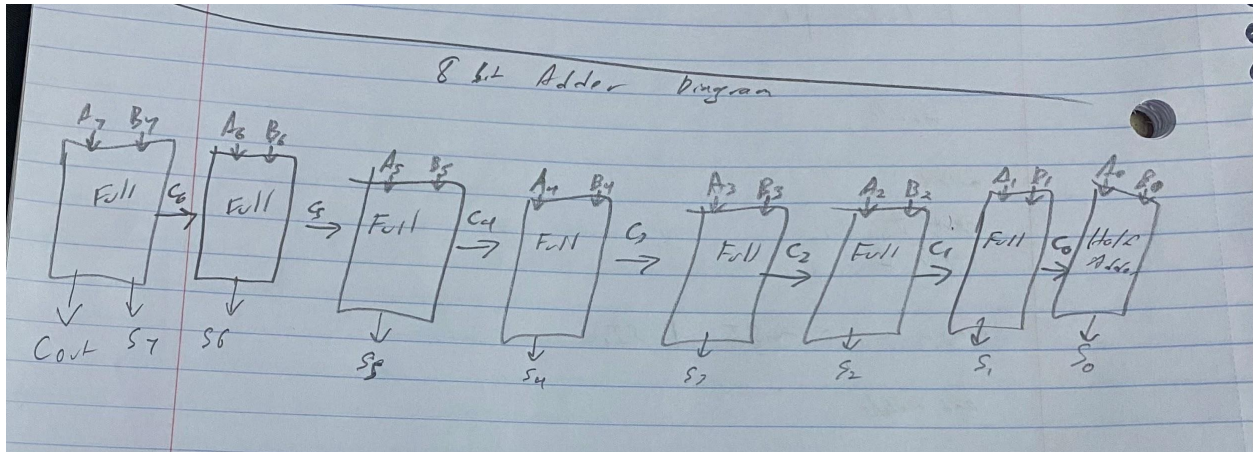
Behavioral simulation:

Results: The output of the experiment matched the intended truth table I shared above in the design. Here are three pictures of some outputs tested:

x= 001, y=111,(First image) .The second image is the same summation but with x and y having swapped values. The last image is x=111, y=111.



Conclusion: The experiment was successful, giving the correct outputs matching our truth table.
8 bit adder diagram:



Using the brute force method for the 8-bit adder would be tedious in the fact that we would have 8 equations to derive. An advantage is that the adder is fast. A disadvantage is the design is tedious to derive. I have designed and applied all necessary parts to this experiment. The design should be tested by checking if the combinations match the desired carries and outputs, as specified by our truth table.